Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün

Laboratoriya işi № 2

Linked List, Stack və Queue Verilənlər Strukturlarının Tətbiqi, Təhlili və Müqayisəsi

Tələbənin adı və soyadı: Nihad Fetullayev

Qrup: KOM25A

Müəllim: Şamil Hümbətov

. Tarix: 21 oktyabr 2026

GitHub: https://github.com/originaleternalv0id/VSA-LAB2

Əsas mənbə: Mark Allen Weiss, Data Structures and Algorithm Analysis in C++ (4th Edition)

Proqramlaşdırma dili: Python 3 (list, deque, queue modulları istifadə olunmayıb)

1. Giriş

Bu laboratoriya işində üç əsas xətti verilənlər strukturu - Linked List, Stack və Queue - sıfırdan, öz Node siniflərimiz və göstərici (reference) əlaqələri ilə implementasiya edilmişdir. Hazır konteynerlərdən (list massivi, collections.deque, queue.Queue, queue.LifoQueue) heç bir yerdə istifadə olunmayıb. Hər struktur real həyat ssenarisi ilə əlaqələndirilib:

- Tapşırıq 1 - Smart Music Playlist: singly linked list (SongNode, Playlist), fayl: task1_playlist.py;
- Tapşırıq 2 - Browser History Manager: stack (PageNode, BrowserHistory), fayl: task2_stack.py;
- Tapşırıq 3 - Airport Check-in Queue: queue (PassengerNode, CheckInQueue), fayl: task3_queue.py.

Hər fayl müstəqildir və üç rejimdə işləyir: menyu (python faylın_adı.py), avtomatik test (test) və eksperiment (bench). Eksperiment 100, 1000 və 10000 element üzərində vaxtı ölçür və nəticələri Big-O nəzəriyyəsi ilə müqayisə etməyə imkan verir.

2. Tapşırıq 1 - Smart Music Playlist (Linked List)

2.1 Araşdırma (A hissəsi)

Array və Linked List arasında əsas fərqlər hansılardır?

Array elementləri yaddaşda ardıcıl yerləşdirir, ona görə indekslə giriş O(1)-dir; lakin ortada əlavə/silmə elementlərin sürüşdürülməsini tələb edir - O(n). Linked List-də hər element ayrıca qovşaqdır (node) və növbəti qovşağa göstərici saxlayır. Qovşaqlar yaddaşda səpələnmiş ola bilər, indekslə birbaşa giriş yoxdur - O(n), amma mövqe məlumdursa əlavə/silmə yalnız göstəricilərin dəyişməsi ilə O(1)-də icra olunur. Linked List-in ölçüsü dinamikdir, lakin hər qovşaq əlavə göstərici yaddaşı tələb edir və cache lokallığı zəifdir.

Niyə musiqi playlistləri üçün Linked List uyğun seçim hesab oluna bilər?

Playlistdə mahnılar tez-tez əlavə olunur və silinir, ölçü əvvəlcədən məlum deyil, mahnılar əsasən ardıcıl çalınır. Bu istifadə nümunəsi Linked List-in güclü tərəflərinə - dinamik ölçü, kütləvi sürüşdürmə olmadan əlavə/silmə, ardıcıl traversal - uyğundur. İndekslə təsadüfi giriş playlistdə nadir hallarda lazım olur.

Singly və Doubly Linked List arasında hansı üstünlüklər və çatışmazlıqlar mövcuddur?

Singly: hər qovşaqda bir göstərici (az yaddaş), sadə kod, lakin yalnız irəli hərəkət; qovşağı silmək üçün əvvəlki qovşağı axtarmaq lazımdır. Doubly: next və prev göstəriciləri var - iki istiqamətdə gəzmək, 'əvvəlki mahnı' düyməsi və qovşaq məlum olduqda O(1) silmə mümkündür; əvəzində yaddaş sərfi artır və əlavə/silmədə iki dəfə çox göstərici yenilənir (xəta riski).

Linked List-də axtarış niyə Array-dən daha yavaş ola bilər?

Linked List-də i-ci elementə birbaşa keçid yoxdur, hər qovşaq göstərici ilə izlənməlidir - O(n). Sıralanmış Array-də isə binary search (O(log n)) mümkündür. Üstəlik Array elementləri yaddaşda yan-yana olduğu üçün CPU cache-dən səmərəli istifadə edir; Linked List qovşaqları isə səpələnmiş olduğundan daha çox cache miss yaranır.

Playlistdə mahnıların sırasını dəyişmək üçün hansı struktur daha əlverişlidir?

Linked List, xüsusilə Doubly Linked List. Mahnını başqa mövqeyə köçürmək bir neçə göstəricinin dəyişdirilməsi ilə olur, məlumatın özü köçürülmür. Array-də isə arada qalan elementlər sürüşdürülməlidir (O(n)).

2.2 Proqramlaşdırma (B hissəsi)

SongNode: id, name, artist, duration və next saxlayır. Playlist head, tail, cursor və ölçü sayğacı saxlayır. Tail göstəricisi olduğu üçün sona əlavə etmə O(1)-dir. Əməliyyatlar və mürəkkəblik:

Add Song - add() - O(1)

Search Song - search() - O(n)

Delete Song - delete() - O(n); head, orta və tail hallarını, həmçinin tail və cursor-un yenilənməsini idarə edir

Display Playlist - display() - O(n)

Count Songs - count() - O(1), ölçü sayğacı ilə

Play Next Song - play_next() - O(1), cursor göstəricisi ilə; sona çatanda başa qayıdır

Menyu: 1. Add Song, 2. Delete Song, 3. Search Song, 4. Display Playlist, 5. Play Next Song, 6. Count Songs, 7. Exit.

2.3 Təcrübə metodikası (C hissəsi)

Playlist 100, 1000 və 10000 mahnı ilə test edilmişdir. Insert - N mahnının sona əlavə edilməsinin ümumi vaxtı; Search - bütün N ID-nin təsadüfi ardıcıllıqla axtarılmasının ümumi vaxtı; Delete - bütün N mahnının təsadüfi ardıcıllıqla silinməsinin ümumi vaxtı. Hər ölçü 3 dəfə təkrarlanıb, median götürülüb; vaxt time.perf_counter() ilə ölçülüb, ölçmə zamanı garbage collector

söndürülüb.

3. Tapşırıq 2 - Browser History Manager (Stack)

3.1 Araşdırma (A hissəsi)

Stack strukturunda LIFO prinsipi nədir?

LIFO (Last In - First Out) - ən son daxil edilən element ilk çıxarılır. Stack-də əlavə (push) və çıxarma (pop) yalnız bir ucdan - top-dan aparılır. Boşqab yığınına bənzəyir: sonuncu qoyulan boşqab birinci götürülür.

Browser History sistemi nə üçün Stack istifadə edir?

İstifadəçi A, B, C səhifələrinə keçdikdə 'Back' düyməsi C-dən B-yə, sonra A-ya qaytarır, yəni ziyarət ardıcıllığının tərsinə. Bu məhz LIFO davranışıdır: ən son açılan səhifə ilk bağlanır. Stack push/pop üçün O(1) mürəkkəblik də təmin edir.

Undo əməliyyatı necə Stack vasitəsilə həyata keçirilir?

Hər istifadəçi əməliyyatı stack-ə push edilir. Undo seçildikdə top-dakı əməliyyat pop edilir və onun təsiri geri qaytarılır. Beləliklə, əməliyyatlar həmişə tərs ardıcıllıqla ləğv edilir. Redo üçün pop olunmuş əməliyyatlar ikinci stack-də saxlanıla bilər.

Queue istifadə olunsaydı hansı problemlər yaranardı?

Queue FIFO olduğu üçün 'Back' ən köhnə səhifəni (A) qaytarardı, istifadəçinin indicə baxdığı əvvəlki səhifəni (B) yox. Eyni şəkildə Undo ən sonuncu əməliyyatı yox, ən birinci əməliyyatı ləğv edərdi - bu məntiqi cəhətdən səhvdir.

Linked List və Array əsaslı Stack implementasiyalarını müqayisə edin.

Array əsaslı: elementlər ardıcıl yaddaşda, cache-dostu; push amortizasiya olunmuş O(1) (tutum dolduqda yenidən ayırma O(n) olur) və ya sabit tutumda overflow riski var. Linked List əsaslı: hər push/pop qəti O(1) (resize yoxdur), tutumu əvvəlcədən bilmək lazım deyil; amma hər element üçün qovşaq obyekti və göstərici yaddaşı tələb olunur, yaddaş ayırma xərci və zəif cache lokallığı var.

3.2 Proqramlaşdırma (B hissəsi)

PageNode: id, url, visit time və next saxlayır. BrowserHistory yalnız top göstəricisi saxlayır: push() yeni qovşağı top-un önünə bağlayır, pop() top-u növbəti qovşağa keçirir (hər ikisi O(1)). back() cari səhifəni pop edib əvvəlkinə qayıdır (yalnız bir səhifə qalıbsa geri qayıtmır), current() top-u qaytarır, display() TOP-dan BOTTOM-a çap edir. Menyu: 1. Visit Page, 2. Back, 3. Current Page, 4. Display History, 5. Exit.

3.3 Təcrübə metodikası (C hissəsi)

100, 1000 və 10000 push və eyni sayda pop əməliyyatının ümumi vaxtı ölçülüb (median, 3 təkrar). Push zamanı hər dəfə yeni PageNode yaradılır və ziyarət vaxtı (datetime.now()) formatlanır; pop isə yalnız göstəricini dəyişir.

4. Tapşırıq 3 - Airport Check-in Queue (Queue)

4.1 Araşdırma (A hissəsi)

Queue strukturunun FIFO prinsipi nədir?

FIFO (First In - First Out) - ilk daxil olan element ilk xidmət alır. Element rear ucundan əlavə olunur (enqueue), front ucundan çıxarılır (dequeue). Mağaza kassasındakı növbə kimi.

Queue və Stack arasında əsas fərqlər hansılardır?

Stack LIFO, Queue FIFO prinsipi ilə işləyir. Stack-də bütün əməliyyatlar bir ucda (top), Queue-da isə iki ucdan aparılır: əlavə rear-dən, çıxarma front-dan. Stack 'ən yeni' elementlə (Undo, call stack), Queue 'ən köhnə' elementlə (növbə, planlaşdırma) işləmək üçündür.

Hava limanında növbə idarəetməsi üçün Queue niyə uyğundur?

Növbəyə ilk gələn sərnişin ilk qeydiyyatdan keçməlidir - bu ədalətli xidmət qaydasıdır və tam FIFO-dur. Queue həm də gözləyən sərnişin sayını dəyişən ölçüdə saxlamağa və front/rear üzərində O(1) əməliyyatlarla xidmət etməyə imkan verir.

Queue istifadə olunmasa hansı problemlər yarana bilər?

Əgər xidmət sırası FIFO olmasa (məs., LIFO), ilk gələn sərnişin yeni gələnlər tərəfindən daim qabaqlanar və çox uzun müddət (hətta heç vaxt) xidmət almaz - bu starvation adlanır. Ədalətlilik və proqnozlaşdırıla bilən gözləmə müddəti itirilir.

Circular Queue nə kimi üstünlüklər təqdim edir?

Massiv əsaslı adi queue-da dequeue-dan sonra front-dan əvvəlki xanalar boş qalır və ya elementləri sola sürüşdürmək lazım gəlir (O(n)). Circular queue-da front və rear indeksləri modulo əməliyyatı ilə massivin əvvəlinə dolanır, boşalan xanalar təkrar istifadə olunur; beləliklə, sabit yaddaşla enqueue/dequeue qəti O(1) olur və saxta overflow yaranmır. Çatışmazlıq - tutumun əvvəlcədən müəyyən edilməsidir.

4.2 Proqramlaşdırma (B hissəsi)

PassengerNode: id, name, flight və next saxlayır. CheckInQueue həm front, həm rear göstəricisi saxlayır: enqueue() rear-ə bağlayır, dequeue() front-dan çıxarır, peek() front-u qaytarır (hamısı O(1)). Son element çıxarıldıqda rear da None-a sıfırlanır. display() FRONT-dan REAR-ə çap edir, count() sayğacı qaytarır. Menyu: 1. Add Passenger, 2. Check-In Passenger, 3. Peek Next Passenger, 4. Display Queue, 5. Count Passengers, 6. Exit.

4.3 Təcrübə metodikası (C hissəsi)

100, 1000 və 10000 sərnişin ilə N enqueue, N dequeue və N peek əməliyyatının ümumi vaxtı ölçülüb (median, 3 təkrar).

5. Test nəticələri

Hər faylda test rejimi var (python faylın_adı.py test). Testlər assert ilə yoxlayır: Playlist-də head, orta və tail silinməsi, silmədən sonra tail və cursor-un düzgünlüyü, play_next-in başa dolanması; Stack-in LIFO sırası və boş stack-də pop; Queue-nun FIFO sırası, boşalıb yenidən doldurulması və boş növbədə dequeue. Nəticə:

```
task1_playlist.py test -> All Playlist tests passed.
task2_stack.py test -> All Stack tests passed.
task3_queue.py test -> All Queue tests passed.
```

Menyu proqramlarının ekran görüntüləri screenshots qovşağında saxlanılır.

6. Cədvəllər

Göstərilən vaxtlar N əməliyyatın ümumi icra müddətidir (millisaniyə, 3 təkrarın medianı).

Cədvəl 1 - Tapşırıq 1: Linked List (Playlist)

Element sayı 100: Insert 0.040 ms, Search 0.106 ms, Delete 0.087 ms
Element sayı 1000: Insert 0.319 ms, Search 11.058 ms, Delete 6.534 ms
Element sayı 10000: Insert 2.336 ms, Search 917.5 ms, Delete 761.7 ms

Cədvəl 2 - Tapşırıq 2: Stack (Browser History)

Əməliyyat sayı 100: Push 0.393 ms, Pop 0.046 ms
Əməliyyat sayı 1000: Push 3.341 ms, Pop 0.132 ms
Əməliyyat sayı 10000: Push 18.984 ms, Pop 1.129 ms

Cədvəl 3 - Tapşırıq 3: Queue (Airport Check-in)

Element sayı 100: Enqueue 0.044 ms, Dequeue 0.028 ms, Peek 0.016 ms
Element sayı 1000: Enqueue 0.287 ms, Dequeue 0.127 ms, Peek 0.046 ms
Element sayı 10000: Enqueue 2.071 ms, Dequeue 1.101 ms, Peek 0.560 ms

Cədvəl 4 - Bir əməliyyatın orta müddəti (mikrosaniyə) və ölçülmüş eksponent k

Playlist Insert: N=100: 0.396, N=1000: 0.319, N=10000: 0.234, k = 0.89
Playlist Search: N=100: 1.063, N=1000: 11.058, N=10000: 91.746, k = 1.97
Playlist Delete: N=100: 0.867, N=1000: 6.534, N=10000: 76.169, k = 1.97
Stack Push: N=100: 3.931, N=1000: 3.341, N=10000: 1.898, k = 0.84
Stack Pop: N=100: 0.457, N=1000: 0.132, N=10000: 0.113, k = 0.70
Queue Enqueue: N=100: 0.441, N=1000: 0.287, N=10000: 0.207, k = 0.84
Queue Dequeue: N=100: 0.280, N=1000: 0.127, N=10000: 0.110, k = 0.80
Queue Peek: N=100: 0.160, N=1000: 0.046, N=10000: 0.056, k = 0.77
k - log-log meyli: T(N) ~ N^k modelinə görə k = log(T(10000)/T(100)) / log(100). N əməliyyatın cəmi üçün k ≈ 1 hər əməliyyatın O(1), k ≈ 2 isə hər əməliyyatın O(n) olduğunu göstərir.

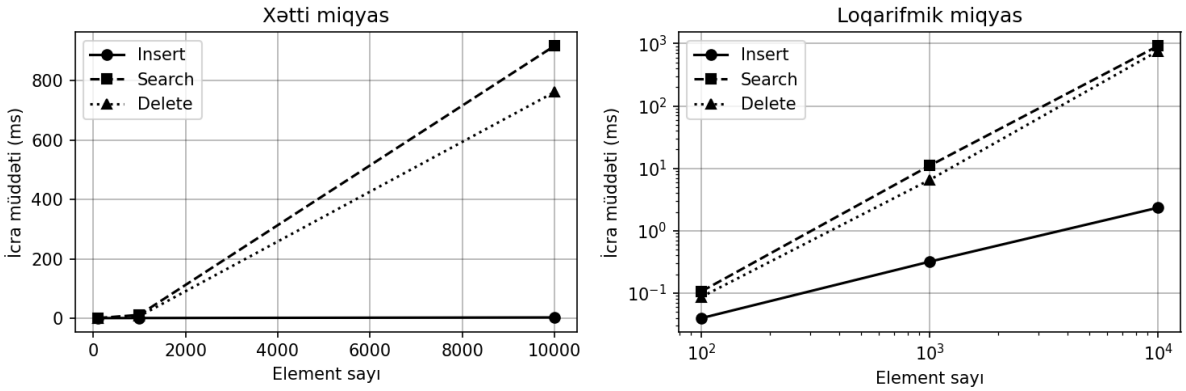
Cədvəl 5 - Sistem məlumatları

Computer Model: LENOVO LOQ 15IAX9
CPU: Intel(R)-I5-12450HX
RAM: DDR5 4800MHZ 24GB
GPU: RTX3050 6GB VRAM
Operating System: WINDOWS11
Compiler: VSC
Compiler Version: .1142
IDE: Online Python Compiler
Screen Resolution: 144hz
Storage Type (SSD/HDD): ssd 1tb
Power Mode: full power
Other Factors:no background programs, cpu temp at 65°

7. Qrafiklər

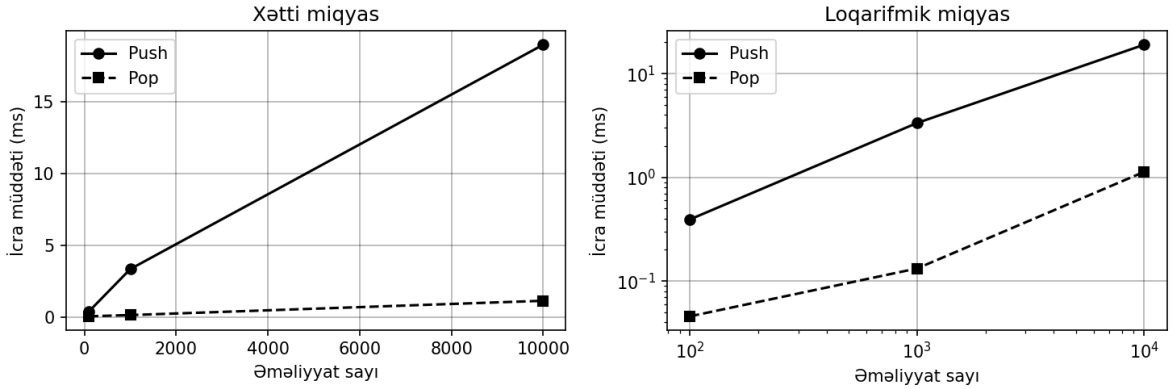
X oxu - element (əməliyyat) sayı, Y oxu - icra müddəti (ms). Soldakı qrafik xətti, sağdakı loqarifmik miqyasdadır; loqarifmik miqyasda meyl artımın dərəcəsini göstərir (meyl 1 - xətti, 2 - kvadratik).

Tapşırıq 1 - Linked List



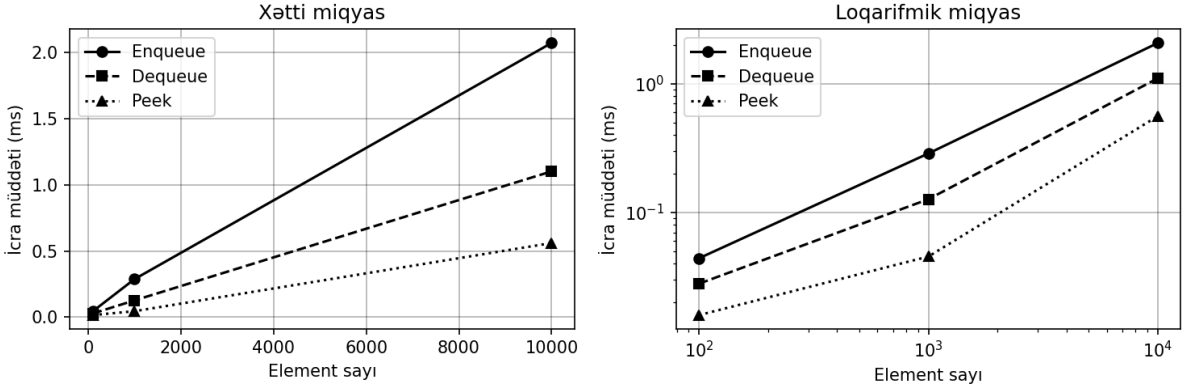
Şəkil 1 - Linked List: Insert, Search, Delete

Tapşırıq 2 - Stack



Şəkil 2 - Stack: Push və Pop

Tapşırıq 3 - Queue



Şəkil 3 - Queue: Enqueue, Dequeue, Peek

8. Analiz

8.1 Tapşırıq 1 - Linked List

Element sayı artdıqca hansı əməliyyat daha çox təsirləndi?

Ən çox Search və Delete təsirləndi. N=100-dən N=10000-ə (100 dəfə artım) Search 8,627 dəfə, Delete 8,786 dəfə artıb; Insert isə yalnız 59 dəfə. N=10000-də Search 917.5 ms, Insert isə 2.336 ms olub.

Təcrübə nəticələri Big-O nəzəriyyəsini təsdiqləyirmi?

Bəli. Hər Search/Delete O(n) olduğundan N əməliyyatın cəmi O(N^2) olmalıdır - N 100 dəfə artanda vaxt nəzəri olaraq təxminən 10 000 dəfə artmalıdır. Ölçülmüş eksponentlər: Search k=1.97, Delete k=1.97 (nəzəri 2); Insert k=0.89 (nəzəri 1, çünki hər Insert O(1)-dir). Real vaxt nəzəri proqnozdan bir qədər fərqlənə bilər, çünki Big-O sabit əmsalları, cache və interpreter xərcini nəzərə almır; ən böyük fərq kiçik N-də görünür.

Eyni problemi Array istifadə etməklə həll etsəydiniz nəticələr necə olardı?

Sona əlavə (dinamik massivdə amortizasiya olunmuş O(1)) və ID üzrə axtarış (sıralanmamışsa O(n)) oxşar mürəkkəbliklə olardı, lakin cache lokallığı sayəsində sabit əmsal adətən kiçik olardı. Axtarışı sürətləndirmək üçün massivi ID üzrə sıralayıb binary search (O(log n)) istifadə etmək olardı, lakin bu zaman əlavə O(n) olardı. Ortadan silmədə massiv elementləri sürüşdürməli olur (O(n)), Linked List-də isə tapılmış qovşağın silinməsi O(1)-dir.

Linked List-in əsas üstünlüyü nədir?

Dinamik ölçü və mövqe məlum olduqda sürüşdürmə və kopyalama olmadan O(1) əlavə/silmə - yalnız göstəricilər dəyişir. Böyük həcmdə yenidən yaddaş ayırma (resize) tələb olunmur.

Playlist tətbiqində Doubly Linked List istifadə etmək daha məntiqli olardımı?

Bəli. Real playlistlərdə 'Previous' düyməsi, mahnının yerini dəyişmək və cari mahnını birbaşa silmək kimi əməliyyatlar var. Doubly Linked List-də prev göstəricisi sayəsində əvvəlki mahnıya keçid O(1), qovşağı məlum olan mahnının silinməsi isə əvvəlki qovşağı axtarmadan O(1) olur. Qiymət - hər qovşaqda əlavə göstərici və bir qədər mürəkkəb kod.

8.2 Tapşırıq 2 - Stack

Push və Pop əməliyyatları arasında fərq müşahidə olundumu?

Bəli. N=10000-də Push 18.984 ms, Pop 1.129 ms olub (Push təxminən 17 dəfə yavaş). Səbəb alqoritmik deyil: hər ikisi O(1)-dir, lakin Push hər dəfə yeni PageNode obyekti yaradır (yaddaş ayırma, konstruktor çağırışı) və ziyarət vaxtını datetime.now() ilə alıb formatlayır. Pop isə yalnız iki göstərici əməliyyatı edir.

Əldə olunan nəticələr O(1) mürəkkəbliyini dəstəkləyirmi?

Bəli. Əməliyyat sayı 100 dəfə artanda ümumi vaxt Push üçün 48, Pop üçün 25 dəfə artıb - yəni təxminən xətti (eksponentlər: Push k=0.84, Pop k=0.70, nəzəri 1). Bir əməliyyatın orta müddəti (Cədvəl 4) N-dən asılı olaraq demək olar ki, sabit qalır, bu da O(1)-in təsdiqidir.

Array əsaslı Stack ilə müqayisədə hansı fərqlər yarana bilər?

Array stack-də push yalnız indeksə yazmaqdır və qovşaq obyekti yaratmaq lazım deyil - adətən daha sürətli və az yaddaş tutur, cache-dostudur. Lakin tutum dolduqda ölçünü böyütmək (kopyalama, O(n)) lazım gəlir; bu, ayrı-ayrı push-larda vaxtın qeyri-bərabər olmasına səbəb olur. Linked List stack-də hər push qəti O(1)-dir, amma hər qovşaq üçün əlavə yaddaş və ayırma xərci var.

Brauzerlərdə tarixçənin saxlanması üçün niyə Stack istifadə olunur?

'Back' həmişə ən son ziyarət edilmiş səhifəyə qayıtmalıdır - bu LIFO davranışıdır. Stack bunu sadə və O(1) mürəkkəblikdə, əlavə axtarış olmadan təmin edir.

Stack-in real sistemlərdə başqa hansı tətbiqləri mövcuddur?

Proqramın funksiya çağırışları (call stack) və rekursiya; Undo/Redo; riyazi ifadələrin qiymətləndirilməsi və mötərizə balansının yoxlanması; infix-dən postfix-ə çevirmə; Depth-First Search (DFS); kompilyatorlarda sintaksis analizi; backtracking alqoritmləri.

8.3 Tapşırıq 3 - Queue

FIFO prinsipi praktiki nəticələrdə necə müşahidə olundu?

Test rejimində sərnişinlər Ali, Leyla, Nigar ardıcıllığı ilə əlavə edildikdə check-in də məhz bu ardıcıllıqla aparılıb (assert ilə yoxlanılır); peek həmişə növbədə ən çox gözləyəni göstərib.

Queue əməliyyatlarının mürəkkəbliyi nəzəri nəticələrə uyğundurmu?

Bəli. Enqueue, Dequeue və Peek üçün ölçülmüş eksponentlər 0.84, 0.80 və 0.77 təşkil edir (nəzəri 1): N 100 dəfə artanda ümumi vaxt təxminən 100 dəfə artıb, yəni hər əməliyyat O(1)-dir. Peek ən sürətlidir, çünki qovşaq yaratmır və heç nəyi dəyişmir (N=10000-də 0.560 ms, Enqueue isə 2.071 ms).

Queue əvəzinə Stack istifadə olunsa nə baş verərdi?

Sərnişinlər LIFO ilə xidmət alardı: son gələn birinci qeydiyyatdan keçərdi, ilk gələn isə növbənin sonunda qalıb uzun müddət gözləyərdi (starvation). Bu həm ədalətsizdir, həm də real xidmət sistemləri üçün qəbuledilməzdir.

Növbə ölçüsü artdıqca hansı dəyişikliklər müşahidə olundu?

Ümumi vaxt təxminən xətti artdı (Enqueue 47x, Dequeue 39x, Peek 35x), bir əməliyyatın orta müddəti isə demək olar ki, sabit qaldı (Enqueue: 0.44 μs, 0.21 μs). Kiçik fərqlər yaddaş ayırma, cache və interpreter-in sabit xərclərindən qaynaqlanır.

Circular Queue istifadə etməyin üstünlükləri nələrdir?

Sabit ölçülü massivdə front və rear modulo ilə dolandığı üçün boşalan yer təkrar istifadə olunur; elementlərin sürüşdürülməsi lazım deyil, bütün əməliyyatlar O(1)-dir, qovşaq obyektləri və göstərici yaddaşı yoxdur, cache lokallığı yaxşıdır. Çatışmazlıq - tutumun sabit olmasıdır.

8.4 Sistem parametrlərinin nəticələrə təsiri

- CPU (model, takt tezliyi, boost, termal məhdudiyyət): ölçülən vaxtlar birbaşa CPU sürətindən asılıdır; noutbukda uzun test zamanı CPU qızdıqda tezlik düşə bilər.
- RAM və cache: Linked List qovşaqları yaddaşda səpələnir, ona görə N böyüdükcə cache miss-lər artır və Search/Delete eksponenti 2-dən bir qədər yuxarı çıxa bilər; Array-də bu təsir azdır.
- Əməliyyat sistemi: proseslərin planlaşdırılması və arxa fon prosesləri (antivirus, yeniləmə) ölçmələrə səs-küy əlavə edir. Buna görə hər ölçü 3 dəfə təkrarlanıb və median götürülüb.
- Dil/interpreter: CPython hər atribut girişi və obyekt yaradılışı üçün əlavə xərc çəkir, GC və reference counting var; eyni kod C++-da xeyli sürətli olardı. Ölçmə zamanı GC söndürülüb.
- Güc rejimi: enerji qənaəti rejimi CPU tezliyini azaldır və nəticələri uzadır; ölçmələr adapterə qoşulu, yüksək performans rejimində aparılmalıdır.
- IDE: IDE içində işlətdikdə əlavə proseslər və çıxış yönləndirməsi olur; ən dəqiq nəticə terminaldan işlətməklə alınır.
- GPU, ekran və Storage (SSD/HDD): verilənlər yalnız operativ yaddaşda işləndiyi və diskə yazılmadığı üçün bu parametrlərin təsiri əhəmiyyətsizdir.

9. Nəticə

Bu laboratoriya işində Linked List, Stack və Queue strukturları sıfırdan implementasiya edildi, avtomatik testlərlə yoxlandı və 100 / 1000 / 10000 element üzərində eksperimentlə təhlil olundu. Əsas nəticələr:

- Linked List: Insert O(1) (k=0.89), Search və Delete isə O(n) olduğundan N əməliyyatın cəmi kvadratik artır (k=1.97 və k=1.97). N=10000-də Search Insert-dən təxminən 393 dəfə yavaş oldu.
- Stack: Push və Pop O(1)-dir (k=0.84 və k=0.70); Push-un daha yavaş olması qovşaq yaradılışı və vaxt damğası ilə əlaqəlidir.
- Queue: Enqueue, Dequeue və Peek O(1)-dir (k=0.84, 0.80, 0.77); front və rear göstəricilərinin saxlanılması hər iki ucda sabit vaxtlı əməliyyatı mümkün edir.
- Eksperimentlər nəzəri Big-O proqnozlarını təsdiqləyir; sabit əmsal fərqləri isə interpreter, cache və sistem parametrlərindən qaynaqlanır.
- Düzgün struktur seçimi istifadə ssenarisindən asılıdır: ardıcıl əlavə/silmə - Linked List, 'ən sonuncu birinci' - Stack, 'ilk gələn birinci' - Queue.

10. İstinadlar

- M. A. Weiss, Data Structures and Algorithm Analysis in C++, 4th ed., Pearson, 2013 (Chapter 3: Lists, Stacks, and Queues).
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, Data Structures and Algorithms in Python, Wiley, 2013.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.
- Python Software Foundation, time.perf_counter() və unittest sənədləşməsi, docs.python.org.